Corso di Programmazione Web

Simulazione di esame - Nuova traccia

Questo esame è a libro aperto, ma tutto il lavoro deve essere svolto individualmente.

Rispondere a ogni domanda in modo chiaro, utilizzando descrizioni, tabelle o esempi dove necessario.

Fornire esempi di richieste e risposte per mostrare il funzionamento delle API.

Non scrivere codice effettivo, ma fornire progettazioni API dettagliate, formati di richiesta/risposta e modelli di dati.

Sistema di gestione delle sale studio universitarie

Ci è stato chiesto di progettare un sistema web per la gestione delle sale studio di un'università.

Il sistema deve prevedere:

- Un'API per registrare una nuova sala studio, indicando nome, edificio, piano e numero massimo di posti;
- Un'API per ottenere la lista di tutte le sale studio disponibili;
- Un'API per visualizzare le informazioni di una singola sala studio dato il suo ID;
- Un'API per prenotare un posto in una sala studio, indicando nome, cognome, matricola, data, ora di inizio e ora di fine della prenotazione.

Si osservi che ogni sala studio deve essere identificata in modo univoco da un ID assegnato automaticamente dal server.

La capienza massima della sala indica il numero massimo di studenti che possono prenotare la stessa sala nello stesso intervallo orario.

Uno stesso studente non può avere due prenotazioni sovrapposte nello stesso orario.

Si può assumere un sistema semplificato in cui agli utenti non è richiesta autenticazione.

Domanda 1: Definizione delle API - 15 punti

Definire i modelli di dati per richiesta e risposta. Inserire i tipi di dato per ogni campo ed eventuali vincoli - 3 punti in totale.

Definire gli endpoint API, specificando URL e metodo HTTP, e descriverli testualmente dove necessario - 2 punti per ciascuna API.

Specificare i parametri di richiesta e le risposte previste, fornendo un esempio di richiesta con successo per ognuna - 1 punto per ciascuna API.

Domanda 2: Gestione degli Errori - 5 punti

Una buona API deve gestire efficacemente gli errori.

Descrivere i codici di errore rilevanti, ad esempio 400, 404, 422 e 500, e spiegare quando dovrebbero essere utilizzati nel contesto delle API appena progettate.

Fornire almeno due esempi concreti di risposta di errore.

Domanda 3: Estensione - 3 punti

Supponiamo che il sistema venga usato contemporaneamente da molti studenti durante il periodo degli esami.

Spiega come il server dovrebbe gestire il problema della concorrenza per evitare che due o più prenotazioni simultanee superino la capienza massima della stessa sala nello stesso intervallo orario.

MODELLO SALA STUDIO ⟶ {

"idSala" : int : 1 , // ASSEGNATO DAL SERVER, UNIVOCO

"nome" : str : "EFiX", //OBBLIGATORIO, MAX = 50 CARATTERI

"edificio" : str : "edificio 2", //OBBLIGATORIO, A SCELTA ["edificio 2","edificio H",...]

"piano" : int : 3 , // OBBLIGATORIO, MIN=0, MAX=3 PIANI

"posti" : int : 10 // OBBLIGATORIO, MIN=1, MAX=20 POSTI

}

MODELLO PRENOTAZIONE ⟶ { "idSala" : int : 1 ,

"idPrenotazione" : int : 2 , // ASSEGNATO DAL SERVER

"nome" : str : "Aldo", // OBBLIGATORIO, MIN=1, MAX=30 CARATTERI

"cognome" : str : "Brazorf", // OBBLIGATORIO, MIN=1, MAX=30 CARATTERI

"matricola" : str : "10/96/00219", // OBBLIGATORIO, FORMATO VALIDO NN/MM/XXXXX

"oraInizio" : str : "8:00", // OBBLIGATORIO, FORMATO VALIDO HH:MM

"oraFine" : str : "9:00", // OBBLIGATORIO, FORMATO VALIDO HH:MM, SUCCESSIVA AD oraInizio

"data" : str : "2026-06-03" // OBBLIGATORIO, FORMATO VALIDO YYYY-MM-DD

}

API 1 - REGISTRAZIONE NUOVA SALA

POST /salaStudio ⟶

SI USA POST PERCHÈ L'UTENTE INVIA DATI AL SERVER PER CREARE UNA NUOVA REGISTRAZIONE SALA STUDIO

⟵ 201 CREATED

{

"nome" : str : "EFiX", //OBBLIGATORIO, MAX = 50 CARATTERI

"edificio" : str : "edificio 2", //OBBLIGATORIO, A SCELTA ["edificio 2","edificio H",...]

"piano" : int : 3 , // OBBLIGATORIO, MIN=0, MAX=3 PIANI

"posti" : int : 10 // OBBLIGATORIO, MIN=1, MAX=20 POSTI

}

⟶ {

"idSala" : int : 1 , // ASSEGNATO DAL SERVER

"messaggio" : "REGISTRAZIONE AVVENUTA CON SUCCESSO"

}

API 2 - OTTENERE LISTA SALE STUDIO DISPONIBILI

GET /salaStudio

USIAMO GET PERCHÈ VOGLIAMO LA LISTA DELLE SALE STUDIO DISPONIBILI, NESSUN BODY

⟵ 200 OK

{

"idSala" : int : 1,

"nome" : str : "EFiX", //OBBLIGATORIO, MAX = 50 CARATTERI

"edificio" : str : "edificio 2", //OBBLIGATORIO, A SCELTA ["edificio 2","edificio H",...]

"piano" : int : 3 , // OBBLIGATORIO, MIN=0, MAX=3 PIANI

"posti" : int : 10 // OBBLIGATORIO, MIN=1, MAX=20 POSTI

}

API 3 - VISUALIZZARE INFORMAZIONI SINGOLA SALA

GET /salaStudio/{idSala} ⟶

USIAMO GET PERCHÈ VOGLIAMO LA LISTA SPECIFICA DI UNA SALA STUDIO, NESSUN BODY

⟵ 200 OK

{

"idSala" : int : 1,

"nome" : str : "EFiX", //OBBLIGATORIO, MAX = 50 CARATTERI

"edificio" : str : "edificio 2", //OBBLIGATORIO, A SCELTA ["edificio 2","edificio H",...]

"piano" : int : 3 , // OBBLIGATORIO, MIN=0, MAX=3 PIANI

"posti" : int : 10 // OBBLIGATORIO, MIN=1, MAX=20 POSTI

}

API 4 - PRENOTAZIONE POSTO SALA STUDIO

POST /salastudio/{idSala}/prenotazioni →

SI USA POST PERCHÈ L'UTENTE CREA UNA NUOVA PRENOTAZIONE

← 201 CREATED

```
{
  "nome": str: "Aldo",   // OBBLIGATORIO, MIN=1, MAX=30 CARATTERI
  "cognome": str: "Brazorf", // OBBLIGATORIO, MIN=1, MAX=30 CARATTERI
  "matricola": str: "70/94/00219", // OBBLIGATORIO, FORMATO VALIDO NN/MM/XXXXX
  "oraInizio": str: "8:00",  // OBBLIGATORIO, FORMATO VALIDO HH:MM
  "oraFine": str: "9:00",  // OBBLIGATORIO, FORMATO VALIDO HH:MM, SUCCESSIVA AD oraInizio
  "data": str: "2026-06-03"  // OBBLIGATORIO, FORMATO VALIDO YYYY-MM-DD
}
```

→

```
{
  "idPrenotazione": int: 20,
  "message": "PRENOTAZIONE REGISTRATA CON SUCCESSO"
}
```

- 400 BAD REQUEST → INDICA CHE LA RICHIESTA DEL CLIENT È MALFORMATA, INCOMPLETA O CONTIENE DATI MANCANTI. SI USA QUANDO MANCA IL BODY IN UNA POST, MANCA UN CAMPO OBBLIGATORIO, OPPURE IL CLIENT INVIA UNA STRUTTURA DATI SBAGLIATA

POST /aulestudio →

← 400 BAD REQUEST

```
{
  "nome": str: "Sala Studio 1",
  "piano": int: 2
}
```

→

```
{
  "error": "CAMPI OBBLIGATORI: edificio, posti"
}
```

- 404 NOT FOUND → INDICA CHE LA RISORSA RICHIESTA NON ESISTE. SI USA QUANDO L'UTENTE RICHIEDE UNA SALA INESISTENTE, PROVA A PRENOTARE UNA SALA INESISTENTE, OPPURE RICHIAMA UN ENDPOINT NON VALIDO

GET /aulastudio/99 →

← 404 NOT FOUND

```
{
  "error": "SALA STUDIO CON ID 99 NON TROVATA"
}
```

- 422 UNPROCESSABLE ENTITY → INDICA CHE LA RICHIESTA È SINTATTICAMENTE CORRETTA, MA CONTIENE DATI CHE VIOLANO VINCOLI LOGICI O DI VALIDAZIONE. SI USA QUANDO POSTI È MINORE DI 1, ORAFINE NON È SUCCESSIVA AD ORAINIZIO. SI USA QUANDO POSTI È MINORE DI 1, ORAFINE NON È ORAINIZIO, IL FORMATO DATA/ORARIO NON È VALIDO, LO STUDENTE HA GIÀ UNA PRENOTAZIONE SOVRAPPOSTA, OPPURE LA SALA HA RAGGIUNTO LA CAPIENZA MASSIMA

POST /aulastudio/{idAula}/prenotazioni →

← 422 UNPROCESSABLE ENTITY

```
{
  "nome": str: "Aldo",   // OBBLIGATORIO, MIN=1, MAX=30 CARATTERI
  "cognome": str: "Brazorf", // OBBLIGATORIO, MIN=1, MAX=30 CARATTERI
  "matricola": str: "70/94/00219", // OBBLIGATORIO, FORMATO VALIDO NN/MM/XXXXX
  "oraInizio": str: "8:00",  // OBBLIGATORIO, FORMATO VALIDO HH:MM
  "oraFine": str: "9:00",  // OBBLIGATORIO, FORMATO VALIDO HH:MM, SUCCESSIVA AD oraInizio
  "data": str: "2026-06-03"  // OBBLIGATORIO, FORMATO VALIDO YYYY-MM-DD
}
```

→

```
{
  "error": "L'ORARIO DI FINE DEVE ESSERE SUCCESSIVO ALL'ORARIO DI INIZIO"
}
```

- 500 INTERNAL SERVER ERROR → INDICA UN ERRORE INTERNO LATO SERVER. LA RICHIESTA PUÒ ANCHE ESSERE CORRETTA, MA IL SERVER NON RIESCE A COMPLETARLA. SI USA QUANDO IL DATABASE NON È DISPONIBILE, IL SERVER NON RIESCE A SALVARE UNA SOLA O UNA PRENOTAZIONE, OPPURE AVVIENE UN ERRORE IMPREVISTO DURANTE L'ELABORAZIONE

POST /aulastudio/{idAula}/prenotazioni →

← 500 INTERNAL SERVER ERROR

```
{
  "NOME" : STR : "ALDO",   // OBBLIGATORIO, MIN=1, MAX=30 CARATTERI
  "COGNOME" : STR : "BRAZORF", // OBBLIGATORIO, MIN=1, MAX=30 CARATTERI
  "MATRICOLA" : STR : "40/94/00219", // OBBLIGATORIO, FORMATO VALIDO NN/MM/XXXXX
  "ORAINIZIO" : STR : "8:00",  // OBBLIGATORIO, FORMATO VALIDO HH:MM
  "ORAFINE" : STR : "9:00",  // OBBLIGATORIO, FORMATO VALIDO HH:MM, SUCCESSIVA AD ORAINIZIO
  "DATA" : STR : "2026-06-03"  // OBBLIGATORIO, FORMATO VALIDO YYYY-MM-DD
}
```

→

```
{
  "ERROR" : "ERRORE INTERNO DEL SERVER. RIPROVA PIÙ TARDI"
}
```

DOMANDA 3: CONCORRENZA

Durante il periodo degli esami molti studenti possono inviare richieste di prenotazione nello stesso momento. Il server deve evitare che richieste simultanee superino la capienza massima della stessa sala nello stesso intervallo orario.

La soluzione corretta e' controllare la disponibilita' immediatamente prima del salvataggio e gestire controllo e salvataggio come un'unica operazione atomica o comunque sequenziale. Il server verifica quante prenotazioni esistono gia' per quella sala, data e fascia oraria; se ci sono ancora posti disponibili salva la nuova prenotazione, altrimenti restituisce un errore, per esempio 422 Unprocessable Entity.

In questo modo si evita che due richieste arrivate nello stesso momento leggano entrambe una disponibilita' libera e vengano salvate entrambe causando il superamento della capienza massima.